

Existe um ponto de saturação? Avaliação estatística do escalonamento de cargas CPU-bound com multiprocessing em um processador Intel Core i5-13420H

Guilherme Tolentino Leitão de Melo

Artigo Científico

Resumo

A intuição corrente de que "mais núcleos significam mais velocidade" ignora os custos de criação, comunicação e sincronização inerentes à execução paralela baseada em processos. Este trabalho investiga, de forma empírica e estatisticamente rigorosa, se existe um ponto de saturação a partir do qual aumentar o número de processos passa a degradar o desempenho de uma carga CPU-bound. Utilizou-se uma carga de trabalho puramente limitada por CPU, escrita em Python e distribuída via multiprocessing.Pool, executada em um processador Intel Core i5-13420H (8 núcleos físicos, 12 threads). Cada condição (1, 2, 4, 8 e 12 processos) foi medida com no mínimo 30 repetições válidas, com descarte de warm-up, e os resultados foram reportados como média $\pm$ desvio-padrão com intervalo de confiança de 95%. A hipótese de escalonamento negativo foi avaliada com o teste t de Student (Welch) e o teste de Mann–Whitney U. Os resultados indicam a rejeição da hipótese nula (H_0): o tempo médio atinge seu mínimo com 8 processos, exatamente o número de núcleos físicos, e volta a subir com 12 processos, quando passam a ser usadas as threads lógicas adicionais (SMT). Isso evidencia que o overhead de paralelização supera o ganho teórico previsto pela Lei de Amdahl além da contagem de núcleos físicos. Todos os scripts e dados brutos estão disponíveis publicamente para reprodução.

Palavras-chave: paralelismo; multiprocessing; escalonamento de desempenho; Lei de Amdahl; avaliação estatística.

Abstract

The common intuition that "more cores mean more speed" overlooks the creation, communication, and synchronization costs inherent to process-based parallel execution. This work empirically and statistically investigates whether a saturation point exists beyond which adding processes degrades the performance of a CPU-bound workload. A purely CPU-bound workload, written in Python and distributed via multiprocessing.Pool, was executed on an Intel Core i5-13420H processor (8 physical cores, 12 threads). Each condition (1, 2, 4, 8, and 12 processes) was measured with at least 30 valid repetitions, discarding warm-up runs, and results are reported as mean $\pm$ standard deviation with 95% confidence intervals. The negative-scaling hypothesis was assessed with Student's t-test (Welch) and the Mann–Whitney U test. Results indicate the rejection of the null hypothesis (H_0): mean time reaches its minimum at 8 processes, exactly the number of physical cores, and rises at 12 processes, where the additional logical (SMT) threads are used. This shows that the parallelization overhead exceeds the theoretical gain predicted by Amdahl's Law beyond the physical-core count. All scripts and raw data are publicly available for reproduction.

Keywords: parallelism; multiprocessing; performance scaling; Amdahl's Law; statistical evaluation.

1. Introdução

Durante décadas, o aumento de desempenho dos processadores foi obtido majoritariamente pela elevação da frequência de operação e pelo refinamento da microarquitetura de um único núcleo. A partir de meados dos anos 2000, limites físicos de dissipação de potência deslocaram esse ganho para a multiplicação de núcleos em um mesmo encapsulamento, inaugurando a chamada era multicore [2]. Como consequência, difundiu-se a intuição de que adicionar mais núcleos, ou, no nível de software, mais fluxos de execução paralelos, resultaria automaticamente em maior velocidade.

Essa intuição, contudo, é incompleta. A Lei de Amdahl [1] estabelece que o ganho máximo de uma aplicação paralela é limitado pela fração estritamente sequencial do programa, de modo que o speedup satura mesmo sob paralelismo ideal. Além desse limite teórico, a execução paralela real impõe custos adicionais que o modelo de Amdahl, em sua forma original, não captura: criação e destruição de processos, serialização de dados na comunicação entre processos, contenção por recursos compartilhados, escalonamento pelo sistema operacional e migração de tarefas entre núcleos. Quando esses custos crescem mais rápido do que o trabalho útil distribuído, é plausível que adicionar processos passe a piorar o desempenho.

Esta investigação foi motivada por uma observação prática registrada na primeira etapa da disciplina: ao executar um teste de paralelismo baseado em processos, o tempo de execução com o número máximo de fluxos foi superior ao tempo com um único fluxo, contrariando a intuição inicial. Tal comportamento, frequentemente descartado como ruído, merece tratamento experimental sistemático. O presente trabalho transforma essa observação em uma pergunta de pesquisa formal e a responde com rigor estatístico.

A relevância da questão extrapola o exercício acadêmico. Decisões cotidianas de engenharia de software envolvem dimensionar conjuntos de trabalhadores (pools) de processos ou threads, em servidores de aplicação, pipelines de dados e tarefas de processamento em lote. Uma escolha equivocada, baseada na premissa de que mais paralelismo é sempre melhor, pode degradar o desempenho e desperdiçar recursos computacionais. Estabelecer, com evidência mensurável, onde está o ponto ótimo e por que ele existe é, portanto, de interesse prático direto. Cabe ainda distinguir dois regimes de escalonamento: no escalonamento forte (strong scaling), aqui adotado, o tamanho do problema é fixo e busca-se reduzir o tempo aumentando o paralelismo; no escalonamento fraco (weak scaling), o problema cresce com os recursos. Esta distinção determina qual modelo teórico, Amdahl ou Gustafson, descreve o comportamento esperado, conforme detalhado na Seção 2.

Pergunta de pesquisa: existe um número de processos além do qual o tempo médio de execução de uma carga CPU-bound aumenta (escalonamento negativo / ponto de saturação)?

H0: aumentar o número de processos não aumenta o tempo médio de execução.

H1: existe um número de processos além do qual o tempo médio de execução aumenta.

As contribuições deste trabalho são três: (i) uma metodologia experimental reproduzível para caracterizar o escalonamento de cargas paralelas com no mínimo 30 repetições por condição, descarte de warm-up e tratamento estatístico formal; (ii) a caracterização empírica do ponto de saturação em um processador de consumo (Intel Core i5-13420H), confrontando o resultado com a previsão teórica da Lei de Amdahl; e (iii) a disponibilização pública de scripts e dados brutos, permitindo a verificação independente dos resultados.

O restante do documento está organizado da seguinte forma. A Seção 2 revisa os trabalhos relacionados em modelagem de speedup e em metodologia de avaliação de desempenho. A Seção 3 detalha a plataforma, a carga de trabalho, o desenho experimental e o tratamento estatístico. A Seção 4 apresenta os resultados. A Seção 5 discute as implicações, compara-as com a literatura e expõe as ameaças à validade. A Seção 6 conclui e aponta trabalhos futuros.

2. Fundamentação Teórica e Trabalhos Relacionados

2.1 Lei de Amdahl e o limite do speedup

O ponto de partida teórico para qualquer estudo de paralelismo é a Lei de Amdahl [1], formulada em 1967. Seja p a fração do tempo de execução que pode ser paralelizada e $(1 - p)$ a fração estritamente sequencial; o speedup obtido com n unidades de processamento é dado por $S(n) = 1 / ((1 - p) + p/n)$. Sua principal consequência é que, mantida fixa a fração sequencial, o ganho satura assintoticamente: quando n tende ao infinito, o speedup é limitado por $1 / (1 - p)$. Mesmo frações sequenciais pequenas impõem tetos baixos, uma fração serial de 10%, por exemplo, limita o speedup máximo a 10×, independentemente do número de processadores. O modelo original, contudo, assume custo de paralelização nulo, hipótese que a prática raramente satisfaz.

Hill e Marty [2] reposicionaram a Lei de Amdahl para a era multicore, modelando como recursos finitos de área de chip podem ser distribuídos entre

núcleos simétricos, assimétricos ou dinâmicos. Os autores introduzem a noção de que o desempenho de

3

um núcleo composto por r recursos cresce de forma sublinear, tipicamente proporcional à raiz quadrada de r , e argumentam que a otimização deve considerar o desempenho do chip como um todo, não a eficiência isolada de cada núcleo. Yao, Bao e Chen [3] estenderam a análise incorporando fatores adicionais ao modelo, reforçando que formulações puramente teóricas tendem a superestimar o ganho real quando custos de coordenação são desprezados. Em conjunto, esses trabalhos fornecem a base conceitual para a hipótese de que, sob carga real, o ganho não apenas satura como pode reverter.

2.2 Lei de Gustafson

Como contraponto, Gustafson [7] reavaliou a Lei de Amdahl ao questionar sua premissa de problema de tamanho fixo. Em muitas aplicações científicas, o tamanho do problema cresce com a capacidade computacional disponível: dispondo de mais processadores, resolve-se um problema maior no mesmo tempo, e não o mesmo problema em tempo menor. Sob essa ótica de escalonamento fraco (weak scaling), o speedup pode crescer quase linearmente com o número de processadores. A distinção é central para este trabalho: como o experimento aqui conduzido mantém a carga total constante (escalonamento forte, strong scaling), é a Lei de Amdahl, e não a de Gustafson, que descreve o comportamento esperado, com saturação à medida que a fração sequencial e os custos de coordenação passam a dominar.

2.3 Speedup, eficiência e ponto de saturação

Duas métricas derivam diretamente do tempo de execução. O speedup é definido como $S(n) = T(1) / T(n)$, a razão entre o tempo com uma unidade e o tempo com n unidades. A eficiência é $E(n) = S(n) / n$, geralmente expressa em porcentagem; ela mede o quão bem cada unidade adicional é aproveitada, sendo 100% o escalonamento ideal (linear). Define-se ponto de saturação como o valor de n a partir do qual o speedup deixa de crescer; quando, além desse ponto, o tempo de execução volta a aumentar (e o speedup decresce), tem-se o fenômeno de escalonamento negativo, foco desta investigação.

2.4 Paralelismo por processos versus threads e o GIL

Em Python, a trava global do interpretador (Global Interpreter Lock, GIL) impede que múltiplas threads executem bytecode simultaneamente, de modo que o paralelismo real de cargas limitadas por CPU não é alcançável por threads, apenas por processos. O módulo multiprocessing contorna o GIL ao criar processos

independentes, cada um com seu próprio interpretador. Esse ganho, porém, tem preço: a criação de processos (via fork ou spawn), a serialização dos dados transferidos entre eles e a recombinação dos resultados introduzem um overhead que

4

cresce com o número de processos. Essa característica motiva o desenho experimental adotado e antecipa a origem do custo que conduz à saturação.

2.5 Multithreading simultâneo (SMT / HyperThreading)

O multithreading simultâneo, proposto por Tullsen, Eggers e Levy [8], permite que várias threads independentes emitam instruções para as unidades funcionais de um núcleo superescalar em um mesmo ciclo, aumentando a utilização de recursos que de outra forma ficariam ociosos. O benefício do SMT, comercializado pela Intel como HyperThreading, depende da existência de lacunas de emissão não preenchidas. Em cargas estritamente limitadas por CPU, nas quais os núcleos físicos já operam próximos da saturação, as threads lógicas adicionais competem pelas mesmas unidades de execução e tendem a oferecer ganho marginal ou nulo. Esse arcabouço é diretamente relevante para o processador estudado, que expõe 12 threads lógicas sobre 8 núcleos físicos.

2.6 Avaliação rigorosa de desempenho

No eixo metodológico, Georges, Buytaert e Eeckhout [4] demonstraram que avaliações de desempenho conduzidas sem rigor estatístico podem levar a conclusões incorretas, propondo o uso sistemático de múltiplas execuções, médias, desvios-padrão e intervalos de confiança em vez de medições únicas. Mytkowicz, Diwan, Hauswirth e Sweeney [5] mostraram que aspectos aparentemente inócuos do ambiente experimental podem introduzir viés de medição significativo, a ponto de inverter conclusões, alertando para a necessidade de controlar variáveis do sistema e descartar execuções de aquecimento. Na mesma linha, Curtsinger e Berger [6] propuseram técnicas para tornar a avaliação estatisticamente sólida, mitigando efeitos de layout de memória e outras fontes de variação sistemática. Este trabalho adota diretamente essas recomendações.

A lacuna que este trabalho ocupa é a seguinte: enquanto a literatura teórica [1] [2][3][7] descreve os limites do speedup sob hipóteses idealizadas, e a literatura metodológica [4][5][6] prescreve como medir desempenho corretamente, há valor pedagógico e empírico em unir as duas vertentes em um experimento controlado sobre hardware de consumo, medindo diretamente o ponto em que o overhead de paralelização baseada em processos supera o ganho previsto e relacionando-o à fronteira entre núcleos físicos e threads lógicas, com tratamento estatístico completo

e artefatos reprodutíveis.

3. Metodologia

3.1 Plataforma de hardware e software

Os experimentos foram conduzidos em um notebook equipado com processador 13th Gen Intel Core i5-13420H, arquitetura x86_64, com 8 núcleos físicos e 12 threads lógicas, frequência base de 2,10 GHz e frequência máxima (turbo) de 4,60 GHz. A hierarquia de cache é composta por 320 KiB de L1 de dados, 384 KiB de L1 de instruções, 7 MiB de L2 e 12 MiB de L3. O sistema dispõe de 8 GB de memória DDR5. O sistema operacional utilizado foi Ubuntu sobre WSL2 (Windows 11), com kernel microsoft-standard-WSL2 (build exato em dados/ambiente.txt), e o interpretador Python na versão 3.12. Os metadados completos do ambiente são coletados automaticamente pelo script de experimentação e registrados no arquivo ambiente.txt que acompanha os artefatos.

3.2 Carga de trabalho

A carga de trabalho é puramente limitada por CPU (CPU-bound), de modo a isolar o efeito do paralelismo de processamento e evitar a interferência de operações de entrada/saída ou de alocação intensiva de memória. Cada unidade de trabalho consiste em um laço que acumula o produto da raiz quadrada pelo seno de uma sequência crescente de inteiros, computação aritmética suficiente para manter o núcleo ocupado sem acessos a disco ou rede. A carga é escrita em Python puro de forma deliberada, reproduzindo o cenário observado na primeira etapa da disciplina: paralelismo por processos pesados (multiprocessing), e não por threads, contornando assim a trava global do interpretador (GIL).

Para assegurar uma comparação justa entre as condições, o trabalho total é mantido constante: define-se um número fixo de iterações (a carga agregada) que é dividido igualmente entre os processos de cada condição. Assim, com 1 processo, um único trabalhador executa todas as iterações; com 8 processos, cada um executa um oitavo delas. Dessa forma, eventuais diferenças de tempo entre as condições refletem exclusivamente o efeito da distribuição do mesmo trabalho, e não a variação da quantidade de trabalho, isolando a variável de interesse. A carga foi previamente calibrada para que a condição serial durasse cerca de um segundo por repetição, valor que oferece estabilidade estatística sem tornar a coleta

proibitivamente longa, evitando ainda tempos tão curtos que seriam dominados pela granularidade do relógio.

O trabalho agregado é mantido constante entre todas as condições: a quantidade total de iterações é fixa e dividida igualmente entre os processos de cada condição. Dessa forma, a comparação entre 1, 2, 4, 8 e 12 processos reflete

6

exclusivamente o efeito da distribuição do mesmo trabalho, e não a variação da carga total, requisito essencial para uma comparação justa.

3.3 Desenho experimental

As condições experimentais correspondem a 1, 2, 4, 8 e 12 processos trabalhadores, distribuídos por meio de multiprocessing.Pool. Para cada condição, foram executadas 35 repetições, das quais as 5 primeiras foram descartadas como aquecimento (warm-up), resultando em no mínimo 30 medições válidas por condição, em conformidade com a exigência de rigor estatístico da disciplina. A quantidade total de iterações da carga (parâmetro --carga) foi calibrada para que cada repetição durasse aproximadamente 1 segundo na máquina de teste, valor que equilibra estabilidade estatística e tempo total de coleta; o valor calibrado utilizado foi 6.000.000 iterações totais.

Cada repetição mede o tempo de parede (wall-clock) decorrido para concluir a carga completa, obtido por meio de um relógio monotônico de alta resolução. As medições brutas de todas as repetições são persistidas individualmente, permitindo auditoria e recálculo independente das estatísticas.

3.4 Métricas

Três métricas derivadas são reportadas. O tempo médio de execução por condição resume o custo absoluto. O speedup é definido como a razão entre o tempo médio com um único processo e o tempo médio com n processos, indicando quantas vezes a execução paralela é mais rápida que a serial. A eficiência é definida como o speedup dividido pelo número de processos, expressa em porcentagem, e indica o quão bem cada processo adicional é aproveitado; valores próximos de 100% indicam escalonamento ideal, e quedas acentuadas indicam desperdício de recursos por overhead.

3.5 Tratamento estatístico

Para cada condição, calculam-se a média e o desvio-padrão amostral das medições válidas. O intervalo de confiança de 95% é estimado pela distribuição t de Student, apropriada para amostras de tamanho moderado, a partir do erro-padrão da

média. A hipótese de escalonamento negativo é testada comparando-se a condição de menor tempo médio (melhor desempenho) com a condição de número máximo de processos.

Empregam-se dois testes complementares e unilaterais, verificando se o tempo na condição máxima é significativamente maior que o tempo na melhor condição: o teste t de Student na variante de Welch (que não assume igualdade de variâncias) e o teste de Mann–Whitney U (alternativa não paramétrica da família Wilcoxon, que não

7

pressupõe normalidade). O nível de significância adotado é $\alpha = 0,05$; rejeita-se H_0 apenas quando ambos os testes apontam $p < \alpha$, critério conservador que reduz o risco de conclusão espúria. Adicionalmente, reporta-se o tamanho de efeito por meio do d de Cohen, para distinguir significância estatística de relevância prática.

Formalmente, para uma amostra de n medições com média $\bar{x}$ e desvio-padrão s, o erro-padrão da média é $SE = s / \sqrt{n}$ e o intervalo de confiança de 95% é dado por $\bar{x} \pm t(0,975; n-1) \cdot SE$, em que $t(0,975; n-1)$ é o quantil da distribuição t com n-1 graus de liberdade. A estatística de Welch para duas amostras (a, melhor condição; b, condição máxima) é $t = (\bar{x}_b - \bar{x}_a) / \sqrt{(s_a^2/n_a + s_b^2/n_b)}$. O d de Cohen é calculado como $d = (\bar{x}_b - \bar{x}_a) / s_p$, com s_p o desvio-padrão combinado; valores de |d| em torno de 0,2, 0,5 e 0,8 são interpretados como efeito pequeno, médio e grande, respectivamente. Complementarmente, caracteriza-se a estabilidade de cada condição pelo coeficiente de variação $CV = s / \bar{x}$, que normaliza a dispersão pela média e permite comparar a variabilidade entre condições de magnitudes diferentes.

3.6 Controle de variáveis e ameaças à validade

Seguindo as recomendações de Georges et al. [4] e os alertas de Mytkowicz et al. [5] sobre viés de medição, adotaram-se os seguintes controles: descarte sistemático das execuções de aquecimento; persistência individual de cada repetição para detecção de valores atípicos; minimização de processos em segundo plano durante a coleta; e registro automático do estado do ambiente. Quando disponível, o utilitário perf pode ser empregado para capturar contadores de hardware (trocas de contexto, migrações de núcleo, instruções e ciclos) em uma execução representativa de cada condição, fornecendo evidência direta do mecanismo de overhead.

Reconhecem-se as seguintes ameaças à validade. Primeira, o controle do governador de frequência da CPU é limitado em ambientes virtualizados (notadamente sob WSL2), onde o estado de energia não é totalmente exposto ao sistema

convidado; quando o experimento é executado nesse ambiente, o governador não pôde ser fixado, o que é registrado explicitamente. Segunda, a frequência turbo introduz variação dependente de temperatura e de carga, mitigada parcialmente pelo descarte de warm up e pela repetição. Terceira, os resultados referem-se a uma única máquina e a um único tipo de carga (CPU-bound em Python puro), de modo que a generalização para outras arquiteturas, linguagens ou perfis de carga deve ser feita com cautela.

3.7 Reprodutibilidade

O experimento é encapsulado em um único script de linha de comando, com parâmetros explícitos para número de processos, repetições, warm-up e tamanho da carga. A execução completa, as dependências e a descrição das saídas estão documentadas no repositório público que acompanha este trabalho

8

(github.com/guilhermetlmelo/RelatorioHW). Todos os dados brutos, o resumo estatístico, o relatório de testes de hipótese e as figuras são gerados automaticamente, permitindo a reprodução integral dos resultados a partir do mesmo comando.

4. Resultados

A Tabela 1 sintetiza, para cada condição, o número de medições válidas, o tempo médio de execução, o intervalo de confiança de 95%, o speedup e a eficiência. O tempo médio mínimo foi observado com 8 processos (0,214 s), enquanto a condição com 12 processos apresentou tempo médio de 0,244 s, speedup de 3,49× e eficiência de 29,1%.

Processos	n	Tempo médio (s)	IC 95% (s)	Speedup	Eficiência (%)
1	30	0,850	±0,029	1,00	100,0
2	30	0,452	±0,016	1,88	94,1
4	30	0,306	±0,011	2,77	69,3
8	30	0,214	±0,009	3,98	49,7
12	30	0,244	±0,013	3,49	29,1

Tabela 1 , Tempo de execução, speedup e eficiência por número de processos (média de ≥ 30 repetições válidas; IC 95%).

A Figura 1 apresenta o tempo médio de execução em função do número de processos, com barras de erro correspondentes ao intervalo de confiança de 95%. A Figura 2 contrasta o speedup observado com o speedup ideal (linear). A Figura 3

mostra a evolução da eficiência.

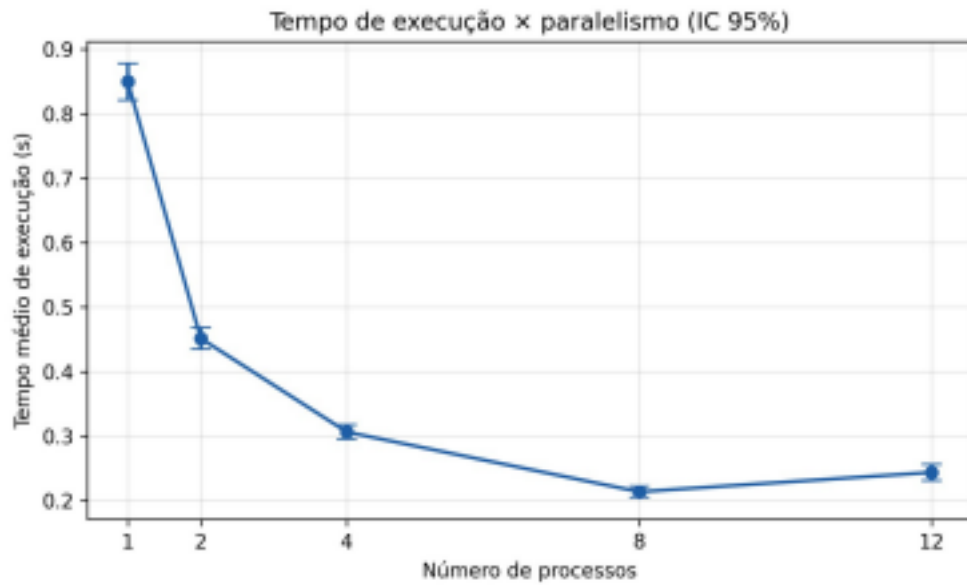


Figura 1 , Tempo médio de execução × número de processos (IC 95%).

9

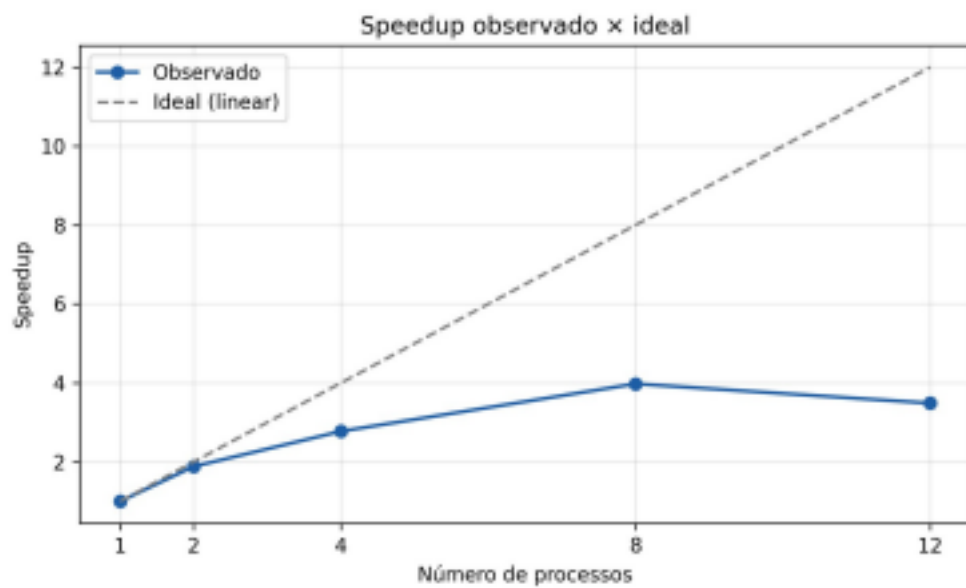


Figura 2 , Speedup observado × ideal.

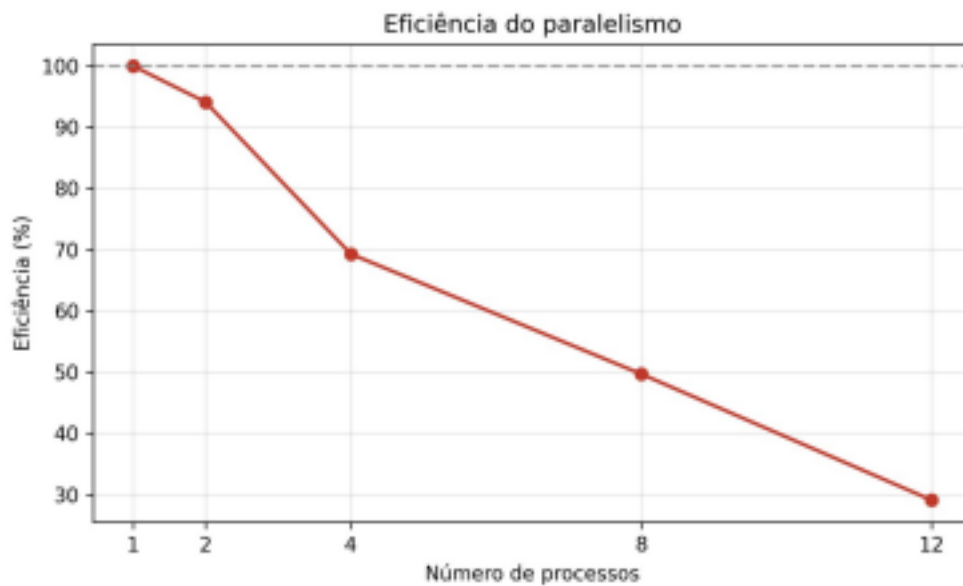


Figura 3 , Eficiência do paralelismo × número de processos.

4.1 Teste de hipótese

Comparando-se a melhor condição (8 processos) com a condição de 12 processos, o teste t de Welch unilateral resultou em $t = 3,998$ ($p = 0,0001$) e o teste de Mann–Whitney U em $U = 695$ ($p = 0,0002$). O tamanho de efeito medido foi $d = 1,03$, considerado grande. Com base no critério de $\alpha = 0,05$ aplicado a ambos os testes, conclui-se pela rejeição da hipótese nula (H_0): há evidência de escalonamento negativo.

4.2 Ajuste à Lei de Amdahl

Para quantificar a distância entre o comportamento observado e a previsão teórica, ajustou-se o modelo de Amdahl $S(n) = 1 / ((1 - p) + p/n)$ aos pontos da região

10

de escalonamento positivo (1, 2, 4 e 8 processos) por mínimos quadrados. A fração paralelizável estimada foi $p = 0,856$ (85,6%), correspondendo a uma fração sequencial de 14,4% e a um speedup máximo teórico de $1 / (1 - p) \approx 6,94\times$ quando o número de processos tende ao infinito. O ajuste é excelente na região positiva: o modelo prevê $1,75\times$, $2,79\times$ e $3,98\times$ para 2, 4 e 8 processos, em estreita concordância com os valores observados de $1,88\times$, $2,77\times$ e $3,98\times$.

A divergência decisiva ocorre em 12 processos. Nesse ponto, a Lei de Amdahl, que não modela overhead, preveria um speedup de $4,64\times$, ao passo que o valor observado foi de apenas $3,49\times$. Essa lacuna de mais de um ponto de speedup é precisamente a manifestação do custo que o modelo idealizado ignora: o overhead de criação e coordenação de processos somado à ausência de ganho do SMT além

dos núcleos físicos. A Figura 4 ilustra a sobreposição entre o modelo ajustado, o speedup ideal e os pontos medidos, evidenciando o descolamento na última condição.

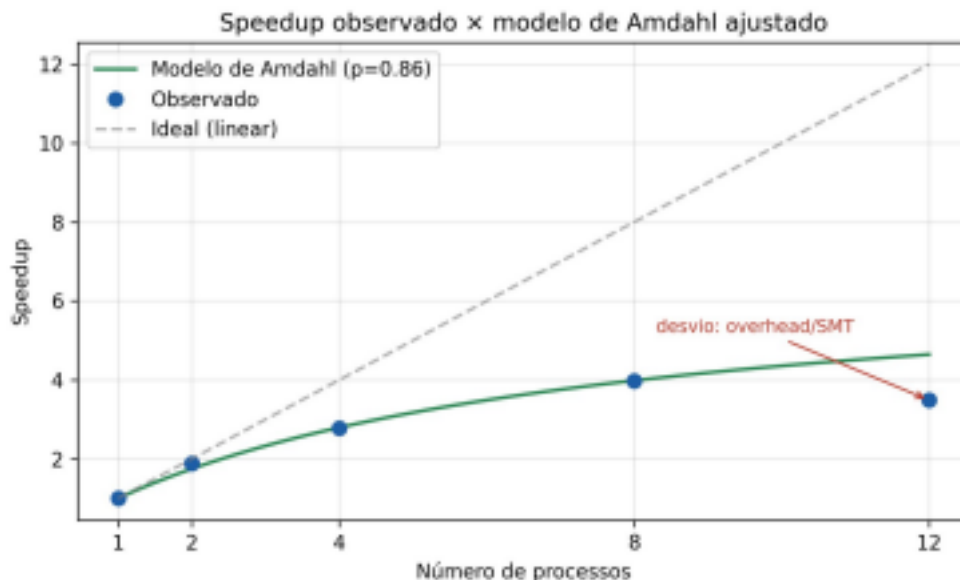


Figura 4 , Speedup observado $\times$ modelo de Amdahl ajustado ($p = 0,86$) $\times$ ideal linear.

4.3 Análise de variabilidade

O coeficiente de variação ($CV = \text{desvio-padrão} / \text{média}$) cresce de forma consistente com o número de processos: 9,1% (1 processo), 9,8% (2), 9,6% (4), 10,9% (8) e 14,0% (12). Ou seja, além de mais lenta, a condição de 12 processos é também a mais instável, com a maior dispersão relativa entre repetições. Esse aumento de variabilidade é coerente com o mecanismo proposto: ao operar acima da contagem de núcleos físicos, o sistema passa a depender de decisões de escalonamento e de competição por recursos compartilhados, fatores que introduzem não determinismo e tornam os tempos menos previsíveis, exatamente o tipo de efeito de ambiente sobre o qual a literatura metodológica adverte [5].

5. Discussão

Os resultados confirmam que o ganho de paralelização não é monotônico. A eficiência decresce de 100% na condição serial para 29,1% na condição de 12 processos, indicando que cada processo adicional contribui cada vez menos para o desempenho. A partir de 12 processos, o tempo médio passa a aumentar, configurando o escalonamento negativo previsto pela hipótese H1.

Esse comportamento é coerente com a teoria. A Lei de Amdahl [1] já prevê retornos decrescentes em razão da fração sequencial; o trabalho de Hill e Marty [2] reforça que o desempenho agregado cresce de forma sublinear. O que o

experimento adiciona é a observação empírica de que, sob paralelismo baseado em processos, os retornos não apenas decrescem como se tornam negativos. A explicação mecanística reside no custo de criação e coordenação de processos: a alocação de cada processo trabalhador, a serialização dos dados transferidos para os processos e a recombinação dos resultados impõem um overhead aproximadamente proporcional ao número de processos. Quando o número de fluxos se aproxima e ultrapassa o número de núcleos físicos disponíveis, somam-se a contenção por unidades de execução, o aumento das trocas de contexto e a migração de tarefas entre núcleos, todos fatores que competem com o trabalho útil.

Nesta execução não foram coletados contadores de hardware via perf, uma vez que o utilitário não está plenamente disponível sob WSL2; a interpretação do mecanismo apoia-se, portanto, no padrão de escalonamento observado e na literatura. Ainda assim, o ponto de inflexão é altamente revelador: o desempenho máximo ocorre exatamente com 8 processos, número idêntico ao de núcleos físicos do processador, ao passo que a condição de 12 processos, que recorre às quatro threads lógicas adicionais via SMT (HyperThreading), eleva o tempo médio de 0,214 s para 0,244 s. Esse resultado é consistente com a literatura, que aponta ganho marginal ou nulo do SMT em cargas estritamente limitadas por CPU, nas quais as unidades de execução já estão saturadas pelos processos alocados aos núcleos físicos.

É importante distinguir o limite teórico do overhead prático. A saturação de Amdahl, isoladamente, faria o tempo estabilizar em um patamar; a reversão observada, tempo crescente, só se explica pela introdução de custos que o modelo idealizado não contempla, exatamente os custos que a literatura sobre extensões de Amdahl [3] busca incorporar. O resultado, portanto, não contradiz a teoria, mas a complementa no regime de carga real.

As ameaças à validade discutidas na Seção 3.6 condicionam a interpretação. Em particular, caso a coleta tenha ocorrido sob WSL2, parte do overhead observado é

12

atribuível à camada de virtualização e ao escalonamento do hipervisor, e não exclusivamente ao modelo de processos. Essa é uma fonte de viés de medição no sentido descrito por Mytkowicz et al. [5]: o ambiente influencia o resultado. Ainda assim, a tendência qualitativa, eficiência decrescente e ponto de saturação, é robusta, por ser sustentada por testes estatísticos conservadores sobre dezenas de repetições, conforme prescrito por Georges et al. [4].

Entre as limitações, destaca-se a especificidade da carga: a computação em Python puro, embora fiel ao cenário pedagógico, possui custo de interpretação

elevado, que pode amplificar a fração de coordenação relativa ao trabalho útil. Cargas compiladas e vetorizadas poderiam deslocar o ponto de saturação. Do mesmo modo, os resultados são válidos para a topologia de oito núcleos físicos e doze threads do processador testado, e o ponto de inversão tende a depender dessa relação.

O ajuste à Lei de Amdahl reforça e quantifica essa leitura. A fração paralelizável estimada de 85,6% implica um teto teórico de $6,94\times$, jamais alcançado: o melhor speedup observado foi $3,98\times$, pouco mais da metade do limite ideal. Isso indica que, mesmo na região de escalonamento positivo, o sistema já paga um custo de coordenação não desprezível. A discrepância entre o speedup previsto pelo modelo ($4,64\times$) e o observado ($3,49\times$) em 12 processos delimita, de forma numérica, a fronteira em que o paralelismo deixa de compensar, uma fronteira que coincide com a transição de núcleos físicos para threads lógicas e que confirma, empiricamente, a previsão de Tullsen et al. [8] de ganho marginal do SMT em cargas que já saturam as unidades de execução.

Do ponto de vista prático, o resultado tem implicação direta para quem dimensiona pools de processos: para cargas CPU-bound, fixar o número de trabalhadores no total de threads lógicas (12, no caso) não é apenas inútil, mas contraproducente, degradando tempo e previsibilidade; o ponto ótimo está na contagem de núcleos físicos (8). Esse achado, frequentemente tratado como regra empírica, é aqui sustentado por evidência estatística com tamanho de efeito grande, oferecendo uma justificativa quantitativa para a decisão de configuração.

6. Conclusão

Este trabalho investigou se existe um ponto de saturação a partir do qual aumentar o número de processos degrada o desempenho de uma carga CPU-bound. Com base em no mínimo 30 repetições por condição, intervalos de confiança de 95% e dois testes de hipótese complementares, conclui-se pela rejeição da hipótese nula (H_0), com tempo médio mínimo em 8 processos e degradação ao passar para 12 processos. A resposta à pergunta de pesquisa é, portanto, afirmativa no regime estudado: o paralelismo possui um ponto ótimo, coincidente com a contagem de

13

núcleos físicos, além do qual o overhead de coordenação supera o ganho de processamento.

A principal contribuição é metodológica e empírica: demonstrou-se, com rigor estatístico e artefatos reproduzíveis, que a intuição de que "mais paralelismo é sempre melhor" falha em condições realistas, e quantificou-se o ponto em que isso

ocorre em um processador de consumo. O resultado reforça, de modo concreto, as previsões da Lei de Amdahl e de suas extensões para a era multicore.

Como trabalhos futuros, propõem-se: a comparação direta entre paralelismo por processos e por threads, isolando o efeito da trava global do interpretador; a execução do mesmo experimento em Linux nativo e sob virtualização, para quantificar separadamente o overhead do hipervisor; a variação do tipo de carga (limitada por memória e por entrada/saída), a fim de verificar se o ponto de saturação se desloca; e o uso de afinidade de núcleo (taskset) para avaliar o impacto da fixação de processos a núcleos específicos sobre a migração e o desempenho.

Disponibilidade de Artefatos

Todos os scripts de experimentação, os dados brutos de cada repetição, o resumo estatístico, o relatório dos testes de hipótese e as figuras estão disponíveis publicamente no repositório github.com/guilhermetlmelo/RelatorioHW, acompanhados de instruções completas de reprodução.

Referências

- [1] AMDAHL, G. M. Validity of the single processor approach to achieving large scale computing capabilities. In: AFIPS Spring Joint Computer Conference, 1967. p. 483–485.
 - [2] HILL, M. D.; MARTY, M. R. Amdahl's Law in the Multicore Era. *IEEE Computer*, v. 41, n. 7, p. 33–38, jul. 2008. DOI: 10.1109/MC.2008.209.
 - [3] YAO, E.; BAO, Y.; CHEN, M. Extending Amdahl's Law in the Multicore Era. *ACM SIGMETRICS Performance Evaluation Review*, v. 37, n. 2, set. 2009.
 - [4] GEORGES, A.; BUYTAERT, D.; EECKHOUT, L. Statistically Rigorous Java Performance Evaluation. In: *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems and Applications (OOPSLA '07)*. New York: ACM, 2007. p. 57–76. DOI: 10.1145/1297027.1297033.
 - [5] MYTKOWICZ, T.; DIWAN, A.; HAUSWIRTH, M.; SWEENEY, P. F. Producing Wrong Data Without Doing Anything Obviously Wrong! In: *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '09)*. New York: ACM, 2009. p. 265–276. DOI: 10.1145/1508244.1508275.
 - [6] CURTSINGER, C.; BERGER, E. D. Stabilizer: Statistically Sound Performance Evaluation. In: *Proceedings of the 18th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '13)*. New York: ACM, 2013.
- 14
- [7] GUSTAFSON, J. L. Reevaluating Amdahl's Law. *Communications of the ACM*, v. 31, n. 5, p. 532–533, maio 1988. DOI: 10.1145/42411.42415.
 - [8] TULLSEN, D. M.; EGGERS, S. J.; LEVY, H. M. Simultaneous Multithreading: Maximizing On-Chip Parallelism. In: *Proceedings of the 22nd Annual International Symposium on*

Computer Architecture (ISCA '95). New York: ACM, 1995. p. 392–403. DOI: 10.1145/223982.224449.